المذكرة الاحترافية

Software Engineering for Machine Learning Engineers

الفصل الرابع: Clean Architecture

تنظيم النظام كله في طبقات، بحيث منطق العمل الأساسي يفضل محميًا من تغيرات التفاصيل التقنية

Layers • Dependency Rule • Separation of Concerns

جدول المحتويات

جدول المحتويات 2
مقدمة الفصل الرابع: من مبادئ التصميم إلى بنية النظام كله 4
٤٫١ Layers — طبقات النظام 5
١. ليه تقسيم النظام لطبقات مهم؟ 5
٢. المشكلة اللي بيحلها تقسيم الطبقات 5
٣. إزاي كانت الشركات بتتعامل قبل تبني تقسيم الطبقات الصارم 5
٤. ليه الطريقة القديمة مش مناسبة 5
٥. الفكرة الأساسية وراء تقسيم الطبقات 5
٦. المفاهيم الأساسية 5
٧. المكونات: مسؤولية كل طبقة 5
٨. إزاي المبدأ ده بيشتغل داخليًا 6
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 6
١٠. فين موقعه داخل مشروع ML حقيقي 6
١١. امتى تلتزم بتقسيم صارم للطبقات 6
١٢. امتى ممكن تتساهل شوية 6
١٣. أفضل الممارسات 6
١٤. أشهر الأخطاء الشائعة 6
١٥. اعتبارات الأداء 6
١٦. الاعتبارات الأمنية 7
١٧. اعتبارات قابلية التوسع 7
١٨. مقارنة مع نهج بديل 7
١٩. مثال عملي صغير 7
٢٠. مثال من مشروع Production حقيقي 7
٢١. ملخص الموضوع 7
٤٫٢ Dependency Rule — قاعدة الاعتماد 9
١. ليه قاعدة الاعتماد مهمة؟ 9
٢. المشكلة اللي بتحلها قاعدة الاعتماد 9
٣. إزاي كانت الشركات بتتعامل قبل الالتزام بقاعدة الاعتماد 9
٤. ليه الطريقة القديمة مش مناسبة 9
٥. الفكرة الأساسية وراء قاعدة الاعتماد 9
٦. المفاهيم الأساسية 9
٧. المكونات 9
٨. إزاي المبدأ ده بيشتغل داخليًا 9
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 10
١٠. فين موقعه داخل مشروع ML حقيقي 10
١١. امتى تلتزم بقاعدة الاعتماد بصرامة 10
١٢. امتى ممكن تتساهل شوية 10
١٣. أفضل الممارسات 10
١٤. أشهر الأخطاء الشائعة 10
١٥. اعتبارات الأداء 10

١٦. الاعتبارات الأمنية 10
١٧. اعتبارات قابلية التوسع 11
١٨. مقارنة مع نهج بديل 11
١٩. مثال عملي صغير 11
٢٠. مثال من مشروع Production حقيقي 11
٢١. ملخص الموضوع 11
٤،٣ Separation of Concerns — فصل الاهتمامات 13
١. ليه فصل الاهتمامات مهم؟ 13
٢. المشكلة اللي بيحلها فصل الاهتمامات 13
٣. إزاي كانت الشركات بتتعامل قبل الالتزام بفصل الاهتمامات 13
٤. ليه الطريقة القديمة مش مناسبة 13
٥. الفكرة الأساسية وراء فصل الاهتمامات 13
٦. المفاهيم الأساسية 13
٧. المكونات 13
٨. إزاي المبدأ ده بيشتغل داخليًا 13
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 14
١٠. فين موقعه داخل مشروع ML حقيقي 14
١١. امتى تطبق فصل الاهتمامات بصرامة 14
١٢. امتى الفصل المفرط يكون ضار 14
١٣. أفضل الممارسات 14
١٤. أشهر الأخطاء الشائعة 14
١٥. اعتبارات الأداء 14
١٦. الاعتبارات الأمنية 14
١٧. اعتبارات قابلية التوسع 15
١٨. مقارنة مع نهج بديل 15
١٩. مثال عملي صغير 15
٢٠. مثال من مشروع Production حقيقي 15
٢١. ملخص الموضوع 15
مشروع عملي على الفصل: إعادة هيكلة نظام ML بطبقات Clean Architecture 17
خطوات التنفيذ 17
تحديات شائعة عند التطبيق 17
إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي 17
ملخص الفصل الرابع وربطه بالفصل القادم 17

مقدمة الفصل الرابع: من مبادئ التصميم إلى بنية النظام كله

في الفصل اللي فات اتعلمنا خمس أنماط تصميم (Design Patterns) بتحل مشاكل متكررة على مستوى الكلاسات والعلاقات بينها.
الفصل ده هياخد نفس الأفكار لمستوى أعلى بكتير: مش تصميم كلاس أو علاقة بين كذا كلاس، وإنما تنظيم المشروع كله كنظام متكامل، عن طريق مبدأ اسمه Clean Architecture.

Clean Architecture مش نمط تصميم واحد، هي طريقة تفكير في تقسيم النظام لطبقات (Layers) واضحة، بحيث كل طبقة ليها مسؤولية محددة، والاعتماد بين الطبقات دي بيتحكم فيه قاعدة صارمة اسمها Dependency Rule - وهي في الحقيقة نفس فكرة Dependency Inversion اللي اتعلمناها في فصل SOLID، لكن مطبقة على مستوى النظام كله مش على مستوى كلاس واحد بس.

ليه بندرس الموضوع ده دلوقتي؟ لأن كل حاجة اتعلمتها لحد دلوقتي - Design Patterns، SOLID، Clean Code - كانت بتحسن جودة الكود على مستوى الكلاس أو الدالة. لكن من غير بنية معمارية واضحة تنظم إزاي الأجزاء دي بتتجمع مع بعض في مشروع كامل، حتى أنضف كود ممكن ينتهي في فوضى تنظيمية على مستوى المشروع.

وهنحتاج المبادئ دي بشكل مباشر في الفصل الجاي (Project Structure)، لأن تنظيم المجلدات والملفات والإعدادات اللي هنشرحه هناك هو التطبيق الحرفي لطبقات Clean Architecture اللي هنتكلم عنها هنا، لكن على مستوى نظام الملفات الفعلي للمشروع.

٤,١ Layers — طبقات النظام

فكرة الطبقات (Layers) هي أساس Clean Architecture: تقسيم النظام لمستويات واضحة، كل مستوى مسؤول عن نوع مختلف من القرارات، من القرارات التقنية التفصيلية للقرارات الجوهرية لمنطق العمل.

١. ليه تقسيم النظام لطبقات مهم؟

من غير تقسيم واضح، منطق العمل الأساسي (زي إزاي بيتم تدريب النموذج أو تقييمه) بيتخلط مع تفاصيل تقنية زي إزاي بيتم الاتصال بقاعدة بيانات أو إزاي بيتم عرض النتيجة لمستخدم. الخلط ده بيخلي أي تغيير في التفاصيل التقنية (زي تغيير قاعدة البيانات) يأثر على منطق العمل الأساسي اللي المفروض يفضل ثابت.

تقسيم النظام لطبقات بيوفر خريطة ذهنية واضحة لأي مهندس جديد: يعرف فورًا فين المفروض يحط كود جديد حسب نوعه (منطق عمل، أم تفاصيل تقنية، أم واجهة مستخدم).

٢. المشكلة اللي بيحلها تقسيم الطبقات

المشكلة هي الأنظمة اللي بتُعرف بـ Big Ball of Mud - نظام من غير بنية واضحة، حيث منطق العمل، والتفاصيل التقنية، وواجهة المستخدم كلهم متداخلين في نفس الملفات والكلاسات، وأي تغيير بسيط بيتطلب فهم النظام كله من الألف للياء.

٣. إزاي كانت الشركات بتتعامل قبل تبني تقسيم الطبقات الصارم

في المشاريع المبكرة، كان شائع كتابة كل شيء في ملف أو مجموعة ملفات بسيطة من غير فصل حقيقي، خصوصًا في نماذج ML الأولية (Prototypes) حيث السرعة في الوصول لنتيجة كانت أهم من التنظيم.

٤. ليه الطريقة القديمة مش مناسبة

من غير طبقات واضحة، النظام بيصبح صعب الفهم مع نموه، وأي تغيير في جزء تقني بسيط (زي تغيير مكتبة معينة) بيهدد بكسر منطق العمل الأساسي، لأن معدش فيه حدود واضحة تحمي منطق العمل من التفاصيل التقنية المتغيرة.

٥. الفكرة الأساسية وراء تقسيم الطبقات

النظام بيتقسم لدوائر متداخلة (Concentric Circles)، حيث الدائرة الداخلية بتحتوي على منطق العمل الأساسي (Business Logic / Entities)، والدوائر الخارجية بتحتوي على تفاصيل تقنية أكتر (قواعد بيانات، واجهات مستخدم، أطر عمل خارجية). كل ما اتجهت للخارج، زاد التفصيل التقني وقل الثبات.

٦. المفاهيم الأساسية

- Entities (أو Domain Layer): القلب الجوهري للنظام، القواعد والمفاهيم الأساسية اللي مستقلة تمامًا عن أي تفصيل تقني.
- Use Cases (أو Application Layer): منطق العمل المحدد للتطبيق، بيوضح إزاي الـ Entities بتتفاعل مع بعض لتحقيق هدف معين.
- Interface Adapters: طبقة بتحول البيانات بين الشكل المناسب للـ Use Cases والشكل المناسب للأدوات الخارجية (قواعد بيانات، واجهات مستخدم).
- Frameworks and Drivers: الطبقة الخارجية اللي بتحتوي التفاصيل التقنية المحددة (قاعدة بيانات معينة، مكتبة معينة، واجهة ويب معينة).

٧. المكونات: مسؤولية كل طبقة

- طبقة الـ Entities: القواعد الجوهرية اللي مستقلة عن أي تطبيق معين، زي تعريف إيه هو نموذج تدريب صالح.
- طبقة الـ Use Cases: تنسيق العمليات المحددة للتطبيق، زي عملية تدريب نموذج جديد أو تقييم نموذج موجود.
- طبقة الـ Interface Adapters: تحويل البيانات بين تنسيقات مختلفة، زي تحويل نتيجة النموذج لتنسيق JSON مناسب للـ API.

- طبقة الـ Frameworks and Drivers: التفاصيل التقنية الفعلية، زي مكتبة معينة للتعامل مع قاعدة البيانات أو إطار عمل ويب معين.

٨. إزاي المبدأ ده بيشتغل داخليًا

كل طبقة بتتواصل بس مع الطبقة المجاورة ليها مباشرة، عن طريق واجهات محددة، مش بشكل عشوائي عبر النظام كله. ده بيخلق حدود واضحة (Boundaries) بتحمي منطق العمل الأساسي جوه الدوائر الداخلية من أي تغيير في التفاصيل التقنية بالدوائر الخارجية.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات الـ ML الكبرى، بيتم الفصل الواضح بين طبقة منطق النمذجة الأساسي (إزاي بيتم تقييم نموذج) وطبقة التكامل مع الأدوات الخارجية (قواعد بيانات، أنظمة تخزين سحابية، واجهات API)، بحيث فريق النمذجة يقدر يركز على تحسين الخوارزميات من غير القلق بتفاصيل البنية التحتية، والعكس صحيح لفريق البنية التحتية.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في الفصل بين طبقة منطق النموذج الأساسي (خوارزمية التدريب والتقييم)، وطبقة تنسيق العملية (Pipeline Orchestration)، وطبقة التكامل مع مصادر البيانات والتخزين، وطبقة الـ API أو الواجهة اللي بتقدم النتائج للمستخدم النهائي.

١١. امتى تلتزم بتقسيم صارم للطبقات

- في أي نظام متوقع يعيش ويتطور لفترة طويلة.
- في أي نظام هيتكامل مع عدة أدوات وتقنيات خارجية متوقع تتغير مع الوقت.

١٢. امتى ممكن تتساهل شوية

- في سكربت تجريبي صغير جدًا (Proof of Concept) هدفه إثبات فكرة بسرعة قبل الاستثمار الكامل.
- لكن لازم خطة واضحة لإعادة الهيكلة بمعايير Clean Architecture لو الفكرة نجحت وهتدخل Production.

١٣. أفضل الممارسات

Best Practices

- خلي منطق العمل الأساسي (Use Cases وEntities) في الدوائر الداخلية، مستقل تمامًا عن أي تفصيل تقني.
- استخدم واجهات واضحة (Interfaces) للتواصل بين الطبقات المختلفة.
- تجنب القفز المباشر بين طبقات غير متجاورة، خلي التواصل يمر عبر الطبقة المجاورة.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- خلط منطق العمل الأساسي مع تفاصيل تقنية محددة (زي استدعاء مكتبة قاعدة بيانات مباشرة جوه منطق التدريب).
- السماح لطبقة داخلية بالاعتماد على طبقة خارجية مباشرة، وده بيخالف قاعدة الاعتماد اللي هنشرحها بالتفصيل في الموضوع الجاي.
- تقسيم مفرط للطبقات في مشروع بسيط جدًا، وده بيزود التعقيد من غير فايدة حقيقية.

١٥. اعتبارات الأداء

التقسيم لطبقات بيضيف تكلفة بسيطة في نقل البيانات بين الطبقات (زي تحويل تنسيق البيانات بين الطبقات المختلفة)، لكن التكلفة دي عادة مهملة مقارنة بفايدة الفصل الواضح والقابلية للصيانة.

١٦. الاعتبارات الأمنية

التقسيم الواضح للطبقات بيسمح بفرض قواعد أمان مختلفة على كل طبقة حسب حساسيتها، زي فرض تحقق صارم من الصلاحيات في طبقة الـ Interface Adapters قبل ما البيانات توصل لمنطق العمل الأساسي.

١٧. اعتبارات قابلية التوسع

النظام المقسم لطبقات واضحة أسهل بكتير في التوسع، لأنك تقدر تستبدل أو تحسن طبقة كاملة (زي تغيير قاعدة البيانات المستخدمة) من غير التأثير على باقي الطبقات، طالما الواجهات بين الطبقات فضلت ثابتة.

١٨. مقارنة مع نهج بديل

البديل هو البنية المعمارية أحادية الطبقة (Monolithic) بمعنى غياب أي تقسيم داخلي)، حيث كل شيء مخلوط في كتلة واحدة. الأسلوب ده أسرع في التطوير الأولي لمشروع صغير، لكنه بيفشل بسرعة مع نمو المشروع وتعدد الفرق، والشركات الكبرى بتستثمر في تقسيم واضح للطبقات من البداية لأي نظام متوقع له عمر طويل.

١٩. مثال عملي صغير

```
# Entities layer - core business rule, independent of any framework
class TrainingExample:
    def __init__(self, features, label):
        self.features = features
        self.label = label

# Use Cases layer - application-specific orchestration
class TrainModelUseCase:
    def __init__(self, model, data_repository):
        self.model = model
        self.data_repository = data_repository

    def execute(self):
        examples = self.data_repository.get_training_examples()
        return self.model.fit(examples)

# Interface Adapters layer - converts external data to the format Use Cases need
class CsvDataRepository:
    def __init__(self, path):
        self.path = path

    def get_training_examples(self):
        raw_rows = read_csv(self.path)
        return [TrainingExample(row['features'], row['label']) for row in raw_rows]
```

٢٠. مثال من مشروع Production حقيقي

في منصة ML كبيرة، طبقة الـ Use Cases بتحتوي على منطق مثل TrainModelUseCase وEvaluateModelUseCase، مستقلة تمامًا عن كون البيانات جايه من S3 أو من قاعدة بيانات أو من API. طبقة الـ Interface Adapters هي المسؤولة عن تحويل البيانات الفعلية من أي مصدر للشكل اللي طبقة الـ Use Cases بتفهمه.

٢١. ملخص الموضوع

خلاصة Layers

- النظام بيتقسم لدوائر متداخلة، من منطق عمل جوهري بالداخل لتفاصيل تقنية بالخارج.
- كل طبقة بتتواصل بس مع الطبقة المجاورة عن طريق واجهات محددة.
- الفصل ده بيحمي منطق العمل الأساسي من تغيرات التفاصيل التقنية.

٤,٢ Dependency Rule — قاعدة الاعتماد

قاعدة الاعتماد هي القانون اللي بيحكم إزاي الطبقات اللي اتكلمنا عنها في الموضوع اللي فات بتتفاعل مع بعض: الاعتماد لازم يكون دايمًا من الخارج للداخل، أبدًا العكس.

١. ليه قاعدة الاعتماد مهمة؟

من غير قاعدة صارمة تتحكم في اتجاه الاعتماد بين الطبقات، سهل جدًا إن منطق العمل الأساسي (الطبقة الداخلية) ينتهي بيه معتمد بشكل مباشر أو غير مباشر على تفاصيل تقنية (الطبقة الخارجية)، وده بيلغي كل فايدة تقسيم الطبقات من الأساس.

قاعدة الاعتماد هي في الحقيقة تطبيق مباشر لمبدأ Dependency Inversion اللي اتعلمناه في فصل SOLID، لكن على مستوى النظام كله بدل كلاس واحد بس.

٢. المشكلة اللي بتحلها قاعدة الاعتماد

المشكلة هي منطق العمل الأساسي اللي بيعتمد مباشرة على تفاصيل تقنية محددة، زي كلاس منطق التدريب اللي بيستدعي مباشرة مكتبة معينة للاتصال بقاعدة بيانات. أي تغيير في التفاصيل التقنية دي بيجبرك تعدل منطق العمل الأساسي، رغم إنه من المفروض يفضل مستقر وثابت.

٣. إزاي كانت الشركات بتتعامل قبل الالتزام بقاعدة الاعتماد

كان شائع كتابة الكود بحيث الطبقات العليا (منطق العمل) بتستدعي مباشرة تفاصيل الطبقات التقنية الأدنى (زي قواعد البيانات والمكتبات الخارجية) من غير أي طبقة تجريد فاصلة بينهم.

٤. ليه الطريقة القديمة مش مناسبة

الاعتماد المباشر من منطق العمل على التفاصيل التقنية بيخلق ترابط صارم (Tight Coupling) بين طبقتين المفروض يكونوا مستقلين تمامًا، وأي تغيير في التفاصيل التقنية (زي تبديل قاعدة بيانات) بيؤثر مباشرة على منطق العمل الأساسي، حتى لو المنطق نفسه ميتغيرش.

٥. الفكرة الأساسية وراء قاعدة الاعتماد

الاعتماد الفعلي في الكود (من ينادي على مين) لازم يكون دايمًا من الطبقات الخارجية للداخلية، لكن اتجاه التحكم (Control Flow) ممكن يكون عكس كده عن طريق واجهات مجردة معرفة في الطبقة الداخلية وبتتنفذ في الطبقة الخارجية - وده بالظبط نفس فكرة Dependency Inversion.

٦. المفاهيم الأساسية

- Source Code Dependency: اتجاه اعتماد الكود الفعلي (مين بيستورد أو يستدعي مين)، لازم يكون من الخارج للداخل بس.
- Control Flow: اتجاه تنفيذ العمليات فعليًا وقت التشغيل، ممكن يكون من الداخل للخارج عن طريق الواجهات المجردة.
- Boundary Interfaces: الواجهات المعرفة في الطبقة الداخلية، وبتتنفذ فعليًا في الطبقة الخارجية، وهي الأداة اللي بتحقق عكس اتجاه الاعتماد.

٧. المكونات

- واجهة مجردة معرفة في الطبقة الداخلية (زي DataRepository Interface في طبقة الـ Use Cases).
- تطبيق فعلي للواجهة دي في الطبقة الخارجية (زي CsvDataRepository في طبقة الـ Interface Adapters).
- حقن التطبيق الفعلي (Dependency Injection) للطبقة الداخلية وقت التشغيل، من غير ما الطبقة الداخلية تعرف تفاصيل التطبيق.

٨. إزاي المبدأ ده بيشتغل داخليًا

الطبقة الداخلية (زي Use Cases) بتعرف واجهة مجردة تحدد إيه اللي محتاجاه من الخارج (زي جلب بيانات التدريب)، من غير ما تعرف إزاي هيتم ده فعليًا. الطبقة الخارجية بتنفذ الواجهة دي بتفاصيلها التقنية الخاصة، وبتتحقن للطبقة الداخلية وقت التشغيل. بكده، الكود الفعلي للطبقة الخارجية هو اللي بيعتمد على تعريف الواجهة الموجود في الطبقة الداخلية، مش العكس.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات الـ ML الكبرى، طبقة منطق التدريب الأساسي بتعرف واجهة DataSource مجردة، والتطبيقات الفعلية (قراءة من S3، من قاعدة بيانات، من Data Warehouse) بتتواجد في طبقة خارجية منفصلة تمامًا، وبتتحقن لمنطق التدريب حسب البيئة (تطوير أو إنتاج) من غير أي تعديل في منطق التدريب نفسه.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في كل نقطة تواصل بين منطق العمل الأساسي والتفاصيل التقنية: واجهة مصدر البيانات، واجهة تخزين النموذج، واجهة إرسال الإشعارات أو تسجيل الأحداث، كلها معرفة في الطبقة الداخلية ومنفذة في الطبقة الخارجية.

١١. امتى تلتزم بقاعدة الاعتماد بصرامة

- في أي نظام متوقع يتعامل مع أكتر من تقنية أو بيئة تشغيل مختلفة (تطوير، اختبار، إنتاج).
- في أي نظام متوقع يتطور منطق العمل الأساسي بيه بمعزل عن التفاصيل التقنية.

١٢. امتى ممكن تتساهل شوية

- في سكربت تجريبي بسيط جدًا هيتشغل مرة واحدة، فيه التطبيق الصارم لقاعدة الاعتماد ممكن يزود تعقيد من غير فايدة حقيقية.

١٣. أفضل الممارسات

Best Practices

- عرف الواجهات المجردة اللي منطق العمل الأساسي محتاجها في الطبقة الداخلية نفسها.
- خلي التطبيقات الفعلية للواجهات دي في الطبقة الخارجية بس.
- استخدم Dependency Injection لحقن التطبيقات الفعلية في نقطة مركزية واحدة (Composition Root).

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- استيراد (Import) مكتبة أو كلاس من طبقة خارجية مباشرة جوه كود الطبقة الداخلية.
- تعريف الواجهة المجردة في الطبقة الخارجية بدل الداخلية، وده بيعكس اتجاه الاعتماد الصحيح.
- تجاهل قاعدة الاعتماد بحجة السرعة، وده بيؤدي تدريجيًا لتآكل الفصل بين الطبقات مع الوقت.

١٥. اعتبارات الأداء

الالتزام بقاعدة الاعتماد بيضيف طبقة استدعاء إضافية بسيطة عبر الواجهات المجردة، لكن التكلفة دي مهملة عمليًا مقارنة بفايدة استقرار منطق العمل الأساسي وسهولة اختباره.

١٦. الاعتبارات الأمنية

فصل منطق العمل عن التفاصيل التقنية بيسمح بتطبيق سياسات أمان مختلفة في كل طبقة خارجية حسب طبيعتها (زي تشفير الاتصال بقاعدة بيانات معينة)، من غير التأثير على منطق العمل الأساسي أو الحاجة لتكرار سياسات الأمان في كل مكان.

١٧. اعتبارات قابلية التوسع

الالتزام الصارم بقاعدة الاعتماد بيسمح بإضافة تطبيقات جديدة للطبقات الخارجية (مصادر بيانات جديدة، خدمات تخزين جديدة) من غير أي تأثير على منطق العمل الأساسي، وده أساس قابلية التوسع طويلة المدى لأي نظام كبير.

١٨. مقارنة مع نهج بديل

البديل هو السماح بالاعتماد المباشر بين الطبقات في أي اتجاه، وهو أسرع في التطوير الأولي لكنه بيؤدي لتآكل تدريجي في الفصل بين الطبقات (Architecture Erosion) مع مرور الوقت، حيث الحدود بين الطبقات بتصبح غير واضحة تدريجيًا. الشركات الكبرى بتفرض قاعدة الاعتماد بصرامة، أحيانًا حتى عن طريق أدوات فحص أوتوماتيكي (Architecture Linting) بتتأكد من عدم مخالفتها.

١٩. مثال عملي صغير

```
# Inner layer defines the abstraction it needs
from abc import ABC, abstractmethod

class DataSource(ABC):
    @abstractmethod
    def get_training_examples(self):
        ...

class TrainModelUseCase:
    def __init__(self, data_source: DataSource, model):
        self.data_source = data_source
        self.model = model

    def execute(self):
        examples = self.data_source.get_training_examples()
        return self.model.fit(examples)

# Outer layer implements the abstraction defined by the inner layer
class S3DataSource(DataSource):
    def __init__(self, bucket, key):
        self.bucket = bucket
        self.key = key

    def get_training_examples(self):
        raw_data = download_from_s3(self.bucket, self.key)
        return parse_training_examples(raw_data)

# Composition root - the only place aware of both layers
use_case = TrainModelUseCase(data_source=S3DataSource('my-bucket', 'data.csv'),
model=model)
```

٢٠. مثال من مشروع Production حقيقي

في منصة ML سحابية حقيقية، طبقة منطق التدريب بتعرف واجهة ModelRegistry مجردة لحفظ النماذج المدربة. في بيئة الاختبار، بيتم حقن تطبيق بسيط بيحفظ في الذاكرة، وفي بيئة الإنتاج بيتم حقن تطبيق بيحفظ في نظام تخزين سحابي حقيقي، من غير أي تعديل في منطق التدريب.

٢١. ملخص الموضوع

> **خلاصة Dependency Rule**

- اعتماد الكود الفعلي لازم يكون دايمًا من الخارج للداخل بس.
- الواجهات المجردة بتتعرف في الطبقة الداخلية، وبتتنفذ في الطبقة الخارجية.
- القاعدة دي هي تطبيق Dependency Inversion على مستوى النظام كله، وبتحمي منطق العمل من تغيرات التفاصيل التقنية.

٤,٣ Separation of Concerns — فصل الاهتمامات

فصل الاهتمامات هو المبدأ الأشمل اللي بيربط كل حاجة اتكلمنا عنها في الفصل ده مع بعض: كل جزء من النظام لازم يهتم بحاجة واحدة بس، وميختلطش مع اهتمامات مختلفة نوعيًا عن اهتمامه الأساسي.

١. ليه فصل الاهتمامات مهم؟

النظام اللي بيخلط أنواع مختلفة من الاهتمامات (منطق عمل، تفاصيل تقنية، عرض بيانات) في نفس المكان بيصبح صعب الفهم والتعديل، لأن أي تغيير في اهتمام واحد بيحتاج تفكيك الاهتمامات التانية المتشابكة معاه عشان تفهم التأثير الكامل للتغيير.

المبدأ ده هو الأساس النظري اللي بُنيت عليه كل الأفكار في الفصل ده: الطبقات (Layers) هي تطبيق لفصل الاهتمامات على مستوى تنظيم النظام، وقاعدة الاعتماد (Dependency Rule) هي الآلية اللي بتحمي الفصل ده من التآكل بمرور الوقت.

٢. المشكلة اللي بيحلها فصل الاهتمامات

المشكلة هي الكود اللي بيخلط اهتمامات مختلفة النوع في نفس المكان، زي دالة واحدة بتحسب دقة النموذج، وفي نفس الوقت بتنسق شكل النتيجة كـ JSON، وفي نفس الوقت بترسلها لخدمة خارجية. كل اهتمام من دول له سبب مختلف تمامًا للتغيير، وخلطهم بيزود احتمال كسر اهتمام لما تعدل في اهتمام تاني.

٣. إزاي كانت الشركات بتتعامل قبل الالتزام بفصل الاهتمامات

كان شائع كتابة دوال أو كلاسات بتؤدي عدة اهتمامات مختلفة في نفس الوقت لتوفير وقت التطوير، خصوصًا في المراحل المبكرة من المشروع حيث السرعة كانت أهم من التنظيم.

٤. ليه الطريقة القديمة مش مناسبة

خلط الاهتمامات المختلفة بيصعب اختبار كل اهتمام بمعزل عن التاني، وبيزود نطاق التأثير لأي تغيير (Blast Radius)، وبيخلي فهم الكود يتطلب استيعاب كل الاهتمامات المتشابكة مرة واحدة بدل التركيز على اهتمام واحد في كل مرة.

٥. الفكرة الأساسية وراء فصل الاهتمامات

كل جزء من النظام (دالة، كلاس، طبقة كاملة) لازم يركز على نوع واحد من القرارات أو الاهتمامات. لو لقيت نفسك بتكتب كود بيتعامل مع منطق العمل وفي نفس الوقت بيتعامل مع تفاصيل تقنية زي تنسيق البيانات لقاعدة بيانات معينة، ده مؤشر واضح إن الاهتمامات دي محتاجة تتفصل.

٦. المفاهيم الأساسية

- Concern: أي جانب من جوانب النظام له سبب مستقل للتغيير، زي منطق العمل، أو طريقة التخزين، أو تنسيق العرض.
- Horizontal Separation: الفصل عبر الطبقات (زي الفصل اللي شرحناه في الموضوع الأول من الفصل ده).
- Vertical Separation: الفصل عبر الميزات أو الوحدات الوظيفية المختلفة (زي فصل ميزة التدريب عن ميزة التقييم عن ميزة النشر).

٧. المكونات

- تحديد الاهتمامات المختلفة الموجودة في جزء معين من الكود.
- فصل كل اهتمام في وحدة مستقلة (دالة، كلاس، أو طبقة كاملة حسب حجم الاهتمام).
- تنسيق التفاعل بين الاهتمامات المفصولة دي عن طريق واجهات واضحة.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما كل اهتمام معزول في وحدته الخاصة، أي تغيير في اهتمام معين بيأثر بس على الوحدة المسؤولة عنه، من غير ما ينتشر لباقي الاهتمامات. ده بيقلل بشكل مباشر من التعقيد الذهني المطلوب لفهم أو تعديل أي جزء من النظام.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في فرق الـ ML الكبرى، الفصل بين اهتمامات مختلفة (منطق النموذج، تنسيق الـ Pipeline، التكامل مع البنية التحتية، عرض النتائج) بيسمح لفرق متخصصة مختلفة تشتغل بالتوازي على نفس المشروع، كل فريق مركز على الاهتمام اللي هو خبير فيه من غير الحاجة لفهم تفاصيل الاهتمامات التانية بعمق.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في كل قرار تنظيمي في المشروع: فصل منطق حساب المقاييس عن منطق عرضها، فصل منطق التدريب عن منطق جدولة التجارب (Experiment Scheduling)، فصل منطق النموذج عن منطق البنية التحتية اللي بيشتغل عليها.

١١. امتى تطبق فصل الاهتمامات بصرامة

- لما النظام كبير بما يكفي إن اهتمامات متعددة فعلًا موجودة ومتشابكة.
- لما فرق مختلفة متوقع تشتغل على اهتمامات مختلفة من نفس النظام.

١٢. امتى الفصل المفرط يكون ضار

- لما النظام بسيط جدًا وفيه اهتمام واحد فعليًا، فالفصل المصطنع لاهتمامات وهمية بيزود التعقيد من غير فايدة.
- لما الفصل بيخلق عدد كبير جدًا من الوحدات الصغيرة لدرجة إن تتبع التفاعل بينهم بقى أصعب من الفهم لو كانوا مجمعين بشكل معقول.

١٣. أفضل الممارسات

Best Practices

- حدد الاهتمامات المختلفة في أي جزء من الكود قبل ما تقرر إزاي تنظمه.
- افصل الاهتمامات حسب سبب التغيير، مش حسب تصنيف تعسفي.
- وازن بين الفصل الكافي والفصل المفرط حسب حجم وتعقيد النظام الفعلي.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- خلط منطق العمل مع تفاصيل العرض أو التخزين في نفس الوحدة.
- الفصل المصطنع لاهتمامات وهمية في مشروع بسيط، وده بيزود التعقيد من غير فايدة.
- تجاهل فصل الاهتمامات تمامًا بحجة السرعة، وده بيؤدي تدريجيًا لنظام Big Ball of Mud.

١٥. اعتبارات الأداء

فصل الاهتمامات بيضيف تكلفة بسيطة في التنسيق بين الوحدات المفصولة، لكن الفايدة في الوضوح وسهولة الصيانة بتفوق التكلفة دي بكتير في أي نظام متوسط أو كبير الحجم.

١٦. الاعتبارات الأمنية

فصل الاهتمامات بيسمح بتطبيق فحوصات أمان مركزة على الاهتمام المسؤول عن التعامل مع البيانات الحساسة أو الوصول الخارجي، من غير الحاجة لتكرار الفحوصات دي في كل مكان بيتعامل مع البيانات.

١٧. اعتبارات قابلية التوسع

الفصل الواضح للاهتمامات بيسمح بتوسيع كل اهتمام بشكل مستقل حسب الحاجة، زي توسيع طبقة التخزين بشكل مختلف تمامًا عن توسيع طبقة منطق العمل، من غير التأثير المتبادل بينهم.

١٨. مقارنة مع نهج بديل

البديل هو دمج الاهتمامات المختلفة في نفس المكان لتوفير وقت التطوير الأولي. الأسلوب ده مقبول لمشروع صغير جدًا أو تجربة سريعة، لكن الشركات الكبرى بتلتزم بفصل الاهتمامات في أي نظام متوقع له نمو وتعقيد حقيقي، لأن تكلفة الفصل لاحقًا بعد التشابك أكبر بكتير من تكلفة الفصل من البداية.

١٩. مثال عملي صغير

```
# Before - concerns mixed together
def evaluate_and_report(model, data):
    accuracy = model.score(data['X'], data['y'])
    report = {'accuracy': accuracy, 'timestamp': time.time()}
    requests.post('https://metrics-service/api', json=report)
    return accuracy

# After - each concern separated into its own unit
def calculate_accuracy(model, data):
    return model.score(data['X'], data['y'])

def build_report(accuracy):
    return {'accuracy': accuracy, 'timestamp': time.time()}

class MetricsReporter:
    def __init__(self, endpoint):
        self.endpoint = endpoint

    def send(self, report):
        requests.post(self.endpoint, json=report)

# Each concern can now be tested and changed independently
accuracy = calculate_accuracy(model, data)
report = build_report(accuracy)
reporter.send(report)
```

٢٠. مثال من مشروع Production حقيقي

في نظام ML كبير، اهتمام حساب المقاييس (Metrics Calculation) منفصل تمامًا عن اهتمام تسجيلها (Metrics Logging)، ومنفصل عن اهتمام عرضها في لوحة متابعة (Dashboard). الفصل ده بيسمح لفريق يحسن طريقة العرض من غير ما يلمس منطق الحساب، والعكس صحيح.

٢١. ملخص الموضوع

خلاصة Separation of Concerns

- كل جزء من النظام لازم يركز على اهتمام واحد وليه سبب واحد للتغيير.

- الفصل بيقلل التعقيد الذهني ونطاق تأثير أي تغيير.
- المبدأ ده هو الأساس النظري اللي بُنيت عليه فكرة الطبقات وقاعدة الاعتماد في الفصل ده كله.

مشروع عملي على الفصل: إعادة هيكلة نظام ML بطبقات Clean Architecture

المشروع ده هدفه إنك تطبق الطبقات وقاعدة الاعتماد وفصل الاهتمامات على نظام ML بسيط، بحيث تشوف إزاي المفاهيم التلاتة بتتكامل في بنية معمارية واحدة متماسكة.

السيناريو: عندك نظام تدريب وتقييم نموذج، منطق التدريب فيه بيستدعي مباشرة مكتبة قراءة ملفات محددة، ومنطق التقييم بيرسل النتيجة مباشرة لخدمة خارجية معينة. عايز تعيد هيكلة النظام ده بطبقات واضحة.

خطوات التنفيذ

- حدد طبقة الـ Entities: عرف الكيانات الجوهرية زي TrainingExample وEvaluationResult، مستقلة تمامًا عن أي تفصيل تقني.
- حدد طبقة الـ Use Cases: ابني TrainModelUseCase وEvaluateModelUseCase، وخليهم يعتمدوا على واجهات مجردة بس (DataSource، MetricsReporter).
- طبق قاعدة الاعتماد: عرف الواجهات المجردة دي جوه طبقة الـ Use Cases نفسها، مش في طبقة خارجية.
- ابني طبقة الـ Interface Adapters: نفذ الواجهات دي بتفاصيل تقنية فعلية (CsvDataSource، HttpMetricsReporter).
- طبق فصل الاهتمامات: تأكد إن كل وحدة (Use Case أو Adapter) بتركز على اهتمام واحد بس، وميختلطش منطق الحساب مع منطق الإرسال أو التخزين.

تحديات شائعة عند التطبيق

- المبالغة في تطبيق Clean Architecture على مشروع صغير جدًا: تقسيم مشروع بسيط لأربع طبقات كاملة بيزود التعقيد من غير فايدة حقيقية. القاعدة العملية: طبق البنية دي بالكامل للأنظمة متوسطة أو كبيرة الحجم، وابسطها للمشاريع الصغيرة.
- الخلط بين قاعدة الاعتماد وحقن الاعتماد: قاعدة الاعتماد هي المبدأ المعماري اللي بيحدد فين تتعرف الواجهات، بينما Dependency Injection هي الآلية العملية لتنفيذ القاعدة دي. الاتنين مرتبطين بشكل وثيق لكنهم مش نفس الحاجة بالظبط.
- صعوبة تحديد حدود الطبقات في مشروع موجود بالفعل: إعادة هيكلة نظام قائم بالفعل من غير طبقات واضحة لبنية Clean Architecture عملية تدريجية، مش تغيير فوري، ولازم تتم بحذر مع وجود اختبارات تحمي السلوك الحالي أثناء إعادة الهيكلة.

إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي

Clean Architecture بتظهر في كل قرار تنظيمي كبير في مشروع ML حقيقي: لما تحدد إن منطق النموذج الأساسي لازم يفضل مستقل عن مصدر البيانات الفعلي (Layers وDependency Rule)، ولما تتأكد إن كل جزء من الـ Pipeline بيركز على اهتمام واحد بس (Separation of Concerns).

في الشركات الكبرى، منصات الـ ML الداخلية بتُبنى بالكامل تقريبًا على مبادئ Clean Architecture، وده اللي بيسمح للمنصة تدعم عشرات الفرق المختلفة، كل فريق بيضيف نماذج ومصادر بيانات جديدة من غير الحاجة لفهم أو التأثير على البنية التحتية الأساسية للمنصة.

ملخص الفصل الرابع وربطه بالفصل القادم

في الفصل ده اتعلمت إزاي تنظم مشروع ML كامل في طبقات واضحة (Layers)، وإزاي تتحكم في اتجاه الاعتماد بين الطبقات دي (Dependency Rule)، وإزاي فصل الاهتمامات (Separation of Concerns) هو المبدأ الأشمل اللي بيربط كل ده مع بعض.

شفنا كمان إزاي القاعدة دي هي في الحقيقة امتداد مباشر لمبدأ Dependency Inversion ونمط Dependency Injection اللي اتعلمناهم في الفصلين السابقين.

الفصل الجاي (Project Structure) هياخد الأفكار المعمارية دي ويترجمها لتنظيم فعلي وملموس: إزاي تنظم المجلدات والملفات (Modular Design وPackages)، وإزاي تدير إعدادات المشروع (Configuration وEnvironment Variables) بحيث تعكس فعليًا الطبقات وحدود الاعتماد اللي اتعلمناها هنا، على مستوى نظام الملفات الحقيقي للمشروع.